Accuracy and Precision Metrics for Classification

1. What You'll Learn in This Section

In this lesson, you'll learn to:

Build a confusion matrix from actual and predicted labels for a binary classifier.
Calculate accuracy and precision from the confusion matrix, by hand and with scikit-learn.
Explain why accuracy can mislead on imbalanced datasets, and why precision exposes what accuracy hides.
Apply a classification threshold to control precision, and compare models using consistent metrics on the same dataset.

2. Detailed Explanation

Accuracy: the baseline metric

Accuracy is a metric for classification that measures overall correctness, regardless of class:

```
Accuracy = (Total number of correct predictions, for any class) / (Total number of instances)
```

Accuracy does not care which class a prediction belongs to — it just checks whether the prediction matched reality. For example, if a dataset has 100 instances and 89 are correctly predicted, accuracy is 0.89, or 89%. This simplicity makes accuracy an intuitive starting point, but as you'll see later in this lesson, it can also hide real problems.

The confusion matrix: where every metric comes from

A confusion matrix is the base structure used to compute accuracy, precision, and (in later lessons) recall. On its own, a confusion matrix isn't a single comparable number — it's detailed internal information about what kind of mistakes a classifier makes and how often. Every other metric in this lesson is *derived from* the values inside it.

In any binary classification problem, one class is designated the positive class — the outcome the system is actually built to detect — and the other is the negative class. For example, in a classifier meant to identify students who will fail a course, "fail" is the positive class (1) because that's the outcome of interest, and "pass" is the negative class (0).

A confusion matrix for binary classification is a 2×2 grid. One common convention is:

Row 1 = actual positive, Row 2 = actual negative
Column 1 = predicted positive, Column 2 = predicted negative

This row/column assignment is just a convention — it can be reversed — but whichever convention is used must be applied consistently, since it changes which cell represents which outcome. The numeric values of the four outcome categories below don't change based on labeling convention; only their position in the grid does.

The four outcome categories:

Cell	Actual	Predicted	Name	Meaning
1	Positive	Positive	True Positive (TP)	Correctly identified positive case
2	Positive	Negative	False Negative (FN)	A positive case incorrectly predicted as negative (a "miss")
3	Negative	Positive	False Positive (FP)	A negative case incorrectly predicted as positive (a "false alarm")
4	Negative	Negative	True Negative (TN)	Correctly identified negative case

The naming logic: the first word (True/False) says whether the prediction matched reality; the second word (Positive/Negative) says which direction the prediction landed in. True Positive and True Negative form the diagonal of desirable, correct predictions. False Negative and False Positive form the opposite diagonal of undesirable, incorrect predictions — a well-performing classifier should have small values in these two cells.

By standard statistical convention, a False Positive is a Type I error, and a False Negative is a Type II error.

Reading the matrix row by row gives per-class insight: the first row (actual positive) shows, out of all truly positive cases, how many were caught (TP) versus missed (FN). The second row (actual negative) shows, out of all truly negative cases, how many were correctly identified (TN) versus wrongly flagged as positive (FP).

Watch out for: cell position is not fixed. Which cell represents TP, FP, FN, or TN depends entirely on how rows and columns are labeled. Always confirm which class is treated as positive versus negative before reading a confusion matrix's cells.

Reading a confusion matrix: worked example with 5 students

Here's a full worked example, for a pass/fail classifier with "fail" as the positive class (1) and "pass" as the negative class (0), on 5 students:

> Actual labels: `0, 0, 1, 0, 1` (students 1, 2, 4 passed; students 3, 5 failed)
> Predicted labels: `0, 0, 0, 1, 1` (students 1, 2, 3 predicted pass; students 4, 5 predicted fail)

Comparing position by position:

Student	Actual	Predicted	Outcome
1	0 (pass)	0 (pass)	True Negative
2	0 (pass)	0 (pass)	True Negative
3	1 (fail)	0 (pass)	False Negative
4	0 (pass)	1 (fail)	False Positive
5	1 (fail)	1 (fail)	True Positive

This gives TP = 1, FN = 1, FP = 1, TN = 2 — summing to 5, the total number of students. The first row (actual positive: students 3 and 5) shows that out of 2 actually-failing students, only 1 (50%) was correctly predicted as failing, and 1 (50%) was missed.

Computing accuracy from the confusion matrix — and where it falls short

Once you have TP, TN, FP, and FN, accuracy has a second, equivalent formula:

```
Accuracy (from confusion matrix) = (TP + TN) / (TP + TN + FP + FN)
```

The numerator is the count of all correct predictions across both classes; the denominator is the total number of instances. Using the pass/fail values above (TP = 1, TN = 2, FP = 1, FN = 1):

```
Accuracy = (1 + 2) / (1 + 2 + 1 + 1) = 3/5 = 0.6 = 60%
```

This matches the simple count: 3 of the 5 predictions were correct.

Accuracy isn't always a safe metric to rely on. Its formula treats every correct prediction equally, regardless of class, so it doesn't disclose bias (class imbalance) in the dataset. When a dataset is imbalanced, accuracy can be misleading about how well a model actually performs on the class that matters most — as the factory examples below will show clearly.

Precision: a sharper lens on the positive class

Precision measures how precise a classifier is when it predicts the positive class. It answers: out of all instances the system predicted as positive, how many are actually positive? Precision deliberately focuses on the positive class only, which makes it a "biased" but highly informative metric when the positive class is the one you actually care about.

```
Precision = TP / (TP + FP)
```

The denominator (TP + FP) is the total number of instances the system predicted as positive, whether that prediction was correct or not.

Using the same pass/fail values (TP = 1, FP = 1):

```
Precision = 1 / (1 + 1) = 0.5 = 50%
```

Compare the two metrics for this dataset: accuracy was 60%, but precision is only 50%. Since the classifier's whole purpose was to catch failing students, precision more accurately exposes that the system is only right half the time when it flags a student as failing — a weaker result than accuracy alone suggests.

Watch out for: precision cannot be computed independently of the confusion matrix. The confusion matrix must be built first, from actual and predicted labels; precision is then derived from its TP and FP cells. The two aren't interchangeable shortcuts for one another.

Precision in action: factory defect detection

A camera on a production line captures an image of each product, and a classifier labels it "defective" or "good." Because missing a defective product damages the company's reputation, "defective" is the positive class and "good" is the negative class. In this context:

> True Positive: predicted defective, actually defective — correctly caught.
> True Negative: predicted good, actually good — desirable, uninteresting case.
> False Positive: predicted defective, actually good — a good item wrongly flagged.
> False Negative: predicted good, actually defective — a miss that slipped through.

Small dataset (20 items): 8 actually defective, 12 actually good. Of the 8 defective items, 6 were correctly caught (TP) and 2 were missed (FN). Of the 12 good items, 9 were correctly identified (TN) and 3 were wrongly flagged (FP):

```
Accuracy  = (TP + TN) / Total = (6 + 9) / 20 = 15/20 = 0.75  → 75%
```

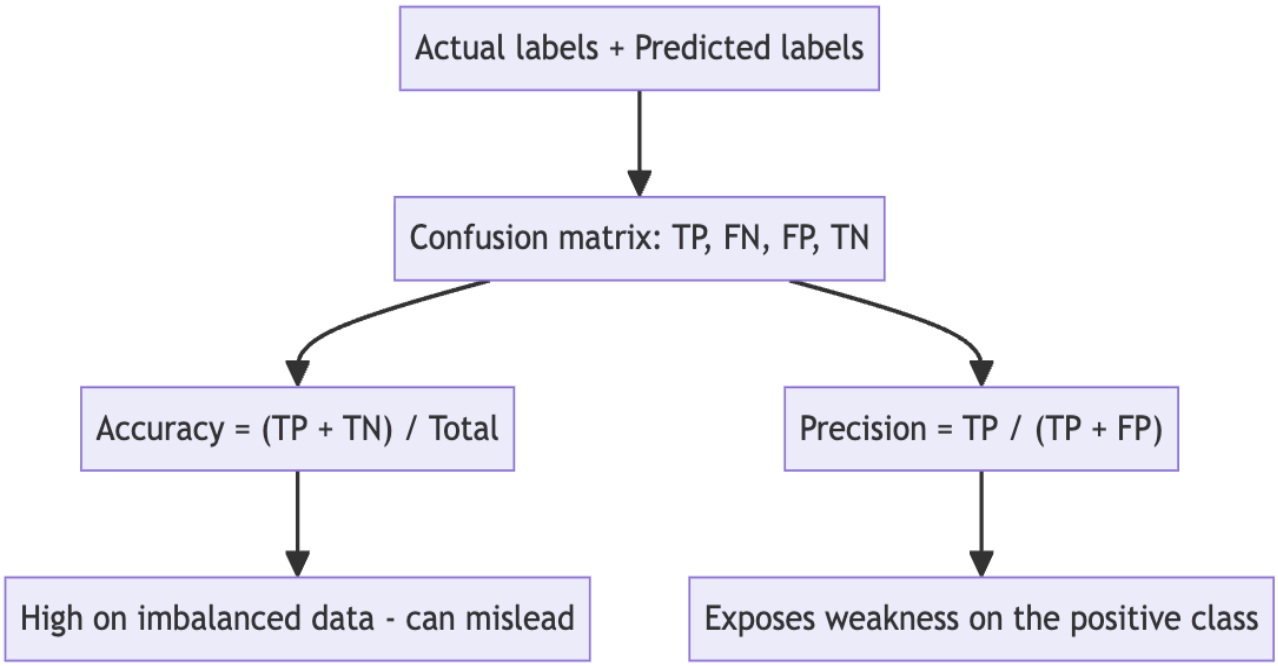
```
Precision = TP / (TP + FP)     = 6 / (6 + 3)   = 6/9    = 0.667 → 66.7%
```

Because this 20-item dataset has some imbalance (12 good vs. 8 defective), accuracy (75%) reads higher than precision (66.7%). Precision more directly discloses how good the system is at the task it was built for.

Scaled-up dataset (500 items, highly biased): representative of a well-run factory where defects are rare — only 15 of 500 items are actually defective, and 485 are actually good. Of the 15 defective items, 12 are caught and 3 are missed. Of the 485 good items, 28 are wrongly flagged and 457 are correctly predicted good:

```
Accuracy  = (TP + TN) / Total = (12 + 457) / 500 = 469/500 = 0.938 → 93.8%
Precision = TP / (TP + FP)     = 12 / (12 + 28)    = 12/40    = 0.3   → 30%
```

This is the central illustration of the whole lesson: accuracy is extremely high (93.8%) purely because the dataset is dominated by the negative (good) class — the model gets most of its correctness "for free." Precision, focused only on the positive class, reveals that only 30% of items labeled defective are actually defective. The accuracy number alone would make this unreliable system look excellent, while precision exposes the real weakness. When a dataset is biased, precision — not accuracy alone — should drive the evaluation.



The classification threshold and its effect on precision

Algorithms such as logistic regression output a continuous classification score between 0 and 1 for each instance, rather than a hard label directly. A default decision threshold of 0.5 is then applied: scores above 0.5 are labeled positive, otherwise negative. For example, five data points scoring 0.7, 0.6, 0.9, 0.2, and 0.3 yield predicted labels positive, positive, positive, negative, negative. Decision trees, by contrast, don't output

this kind of score-based confidence, so their confusion matrix values must come directly from comparing actual and predicted labels rather than from adjusting a threshold.

A borderline case shows why the threshold matters: if a classifier scores an instance at 0.55 (just above the default 0.5), the system isn't very confident, yet it still labels the instance positive. If that instance was actually negative, this becomes a false positive. Raising the threshold (e.g., from 0.5 to 0.6) makes the system require more confidence before predicting positive — borderline cases like the 0.55 example then fall below the new threshold and are predicted negative instead. This eliminates the false positive without changing the count of true positives, and since precision = TP / (TP + FP), reducing false positives while true positives stay the same increases precision.

A visual example of defective ("red") and good ("green") items along a classification-score line makes this concrete:

> At the default threshold (0.5), one green (actually good) item scores just high enough to be classified defective — an extra false positive. Precision here works out to 6/7 = 0.857.
> Raising the threshold to 0.8 removes that borderline green item from the "predicted defective" region entirely. All 3 items above 0.8 are genuinely defective, so false positives drop to zero, and precision = 3/3 = 1.0 (100%).

The general pattern: raising the threshold tends to increase precision, because fewer negative instances slip across the boundary into false-positive territory. The trade-off: a higher threshold can also increase the number of missed positive cases (false negatives).

Watch out for: a model with 100% precision is not automatically a great model. High precision only measures correctness among predicted positives — it says nothing about defective items the system fails to catch at all. In the factory scenario, a good item wrongly pulled off the line damages reputation, so high precision is desirable — but a complete picture of model quality requires pairing precision with recall.

Comparing algorithms using metrics

Evaluation metrics exist to make different models or settings comparable. Two valid kinds of comparison:

> Same algorithm, different settings — e.g., logistic regression run for roughly 500 iterations (setting 1) versus roughly 1,000 iterations (setting 2).
> Different algorithms, same dataset — e.g., comparing logistic regression, decision trees, SVM, or a neural network, all evaluated on the same dataset using the same metrics (such as accuracy and F1 score).

A key rule: comparisons should always be made on the same dataset. Comparing an algorithm's performance across different datasets isn't meaningful, since a difference in results might just reflect a difference in the datasets rather than a genuine difference in algorithm quality. It's fine to report results for the same algorithm across multiple datasets side by side (e.g., accuracy and F1 score for logistic regression on dataset 1, 2, and 3) — but those numbers aren't meant to be compared directly against each other.

Code walkthrough: computing accuracy and precision in Python

The concepts above map directly onto scikit-learn. Setup imports `numpy` as `np`, `pandas` as `pd`, and `matplotlib.pyplot` as `plt`; `LogisticRegression` comes from `sklearn.linear_model`, and `accuracy_score`, `precision_score`, `confusion_matrix`, and `ConfusionMatrixDisplay` come from scikit-learn's metrics utilities. Figure size is set to 7 by 4.5 inches, with a fixed random seed of 42 for reproducibility.

For the 20-item dataset, item numbers, actual labels, and predicted labels (reproducing TP = 6, FN = 2, TN = 9, FP = 3) are stored in a pandas DataFrame with columns `item`, `actual`, `predicted`. The confusion matrix is then built with scikit-learn:

```
cm = confusion_matrix(actual_y, predicted_y, labels=[1, 0])  # 1 = defective, 0 = good
tp, fn, fp, tn = cm.ravel()
print(tp, fn, fp, tn)
```

`confusion_matrix` returns a 2×2 array once actual and predicted label arrays are supplied, with the positive label (defective = 1) and negative label (good = 0) declared explicitly. Because the positive label is listed first in `labels=[1, 0]`, `.ravel()` flattens that array into a single one-dimensional array in the order TP, FN, FP, TN, which is then unpacked into matching variables. The matrix is visualized with `ConfusionMatrixDisplay`, using display labels `["defective", "good"]`, plotted via `.plot()` with the title "Confusion Matrix with 20 Inspected Items."

Manual and built-in calculations agree:

```
manual_accuracy = (tp + tn) / (tp + tn + fp + fn)
print(round(manual_accuracy, 3))          # 0.750

sk_accuracy = accuracy_score(actual_y, predicted_y)
print(round(sk_accuracy, 3))              # 0.750

manual_precision = tp / (tp + fp)
print(round(manual_precision, 3))         # 0.667

sk_precision = precision_score(actual_y, predicted_y)
print(round(sk_precision, 3))             # 0.667
```

This confirms there's no harm in relying on scikit-learn — in fact, its built-in functions are recommended for production code, since manual computation is more error-prone and mainly useful as a learning exercise. The same structure recreated for the 500-item biased dataset gives `sk_accuracy` = 0.938 and `sk_precision` = 0.3, numerically confirming the earlier manual calculation.

A built-in `load_breast_cancer` dataset (from `sklearn.datasets`) demonstrates threshold variation on a more realistic, better-balanced dataset: 569 total instances, 212 malignant and 357 benign. "Malignant" is the positive class. Preparation functions `fit_transform`, `transform`, and `StandardScaler` are used by name here, with their detail left for later lessons. Testing four threshold values gives:

Threshold	Instances flagged malignant	Accuracy	Precision

0.3	63	0.98	0.98
0.5 (default)	61	slightly lower than 0.98	roughly equal to 0.98
0.7	59	0.97	1.00
0.9	fewer still	0.92	1.00

As the threshold rises, precision climbs toward and then holds at 1.0 — the model becomes so conservative that it produces no false positives at all. Meanwhile accuracy gradually drops, showing that some genuinely malignant cases are being missed as the threshold tightens. Because this dataset is comparatively balanced (569 / 2 ≈ 284, and both class counts sit reasonably close to that midpoint), accuracy stays trustworthy here — unlike the highly imbalanced factory dataset. The degree to which accuracy becomes unreliable is proportional to the degree of class imbalance: the more imbalanced the data, the more precision (or other class-focused metrics) is needed.

Recall and F1 score: completing the picture

Precision is an incomplete metric on its own — it only reveals part of the picture for the positive class. Two additional metrics, also computed from the confusion matrix, complete the evaluation:

Recall, computed as `TP / (TP + FN)`.
F1 score, which combines precision (P) and recall (R) into a single number: `F1 = 2PR / (P + R)`.

Confusion matrix, precision, recall, and F1 score function as a connected group: the confusion matrix is built first from actual and predicted labels, TP/TN/FP/FN are read off it, and precision, recall, and F1 score are all derived from those same four numbers. When a dataset is balanced, accuracy and F1 score tend to be close and don't add much extra information beyond one another. When a dataset is biased, F1 score becomes the more informative combined metric to report alongside precision and recall.

3. Key Takeaways

Accuracy = (TP + TN) / Total counts every correct prediction equally, but hides class imbalance — it can look excellent even when the model performs poorly on the class that matters.
The confusion matrix (TP, FN, FP, TN) is the shared foundation every other metric is derived from; positive/negative class assignment and row/column convention must stay consistent.
Precision = TP / (TP + FP) focuses only on the positive class, exposing weaknesses accuracy can't see — most dramatically on the 500-item factory dataset (93.8% accuracy vs. 30% precision).
Raising the classification threshold generally raises precision by eliminating borderline false positives, at the cost of potentially missing more true positives.

100% precision doesn't mean a perfect model — it says nothing about missed positive cases, which is why precision needs recall (and F1 score) alongside it for the full picture.

Mental model: think of accuracy as a wide-angle photo of the whole classroom, and precision as a zoomed-in shot of just the students the classifier called "failing" — the wide shot can look fine even while the zoomed-in shot reveals the class you actually care about is a mess.